

Network-Architecture-Aware Multiplexer Decomposition for Technology Mapping

Yilu Chen¹, Zhifeng Lin¹, Xiaojian Liang, Zhijie Cai, Xiqiong Bai, and Jianli Chen²

Abstract—The decomposition of multiplexers (MUXes) implies the representation of n -to-1 MUXes in terms of basic logic gates. This decomposition significantly impacts the performance of the subsequent optimization and matching in technology mapping. In this brief, we propose a network-architecture-aware multiplexer decomposition that decomposes an n -to-1 MUX into one of two significantly distinct architectures. Specially, a delay-evaluation-based guidance scheme estimates the total delay of the two architectures and then selects the optimal architecture for decomposing. Experimental results demonstrate that our algorithm outperforms Yosys and Design Compiler.

Index Terms—Logic synthesis, technology mapping, technology decomposition, multiplexer decomposition, multiplexer.

I. INTRODUCTION

TECHNOLOGY mapping consists of four main steps: decomposition, optimization, matching, and covering. In the first step, decomposition, each node in the logic network is represented using basic logic gates, e.g., inverter (INV), two-input AND (AND), two-input OR (OR), 2-to-1 multiplexer (MUX2), et al. [1]. This step is crucial for subsequent optimization and matching [2], [3].

Multiplexers (MUXes) can be used in the fields of adders [4], [5], [6], multipliers [7], [8], and reconfigurable digital systems [9], [10], [11]. An n -to-1 MUX (MUX n) can select information from one of the n data inputs and transfer that input to a single output bit [12]. The challenge of multiplexer decomposition is that, as the n value for MUX n increases, the number of possible decomposition

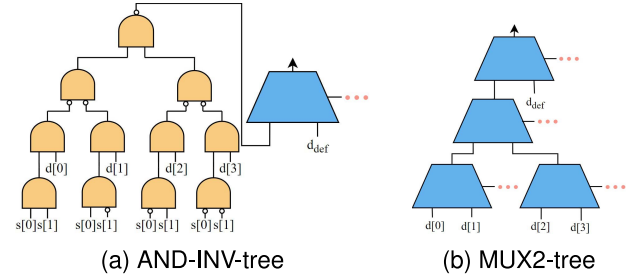


Fig. 1. Different architectures for multiplexer decomposition, where “...” implies some address logic gates for MUX2.

schemes for MUX n also grows exponentially. This problem is NP-complete [13].

Two potential decomposition architectures for MUX n are illustrated in Figure 1. In this brief, the architecture like Figure 1(a) is denoted as AND-INV-tree (AIT), while the architecture like Figure 1(b) is referred to as MUX2-tree (MT). To our best knowledge, most of the works on multiplexers focuses on their hardware implementation [14], [15], with relatively fewer works focusing on logic synthesis. The open-source synthesis tool Yosys [16] utilizes an expression-based technique to decompose MUX n into an AND-OR-INV-tree before sending it to ABC [17]. Thakur et al. [2] proposed an effective method to decompose MUX n into a MUX2-tree, considering the delay of data inputs and address inputs to minimize the overall delay of the MUX2-tree. However, the delay of the network composed of other basic logic gates might be lower, and they did not consider the area and delay of the logic gates. MUX n was decomposed into MUX2s, 4-to-1 MUXes, etc. in [13], [18], but these decomposition methods didn't ultimately result in basic logic gates.

To address the aforementioned problems, in this brief, we propose a network-architecture-aware multiplexer decomposition that decomposes MUX n into one of two significantly distinct architectures, AND-INV-tree and MUX2-tree. Our main contributions are summarized as follows:

- We propose the utilization of two significantly distinct decomposition architectures in multiplexer decomposition. The first one is an AND-INV-tree constructed from the expression of MUX n , while the second one is a MUX2-tree that maintains balance and considers the elimination of identical data inputs.
- We propose an architectural characteristic-based delay evaluation scheme to predict the delay of two architectures and then select the optimal architecture for decomposition.

Manuscript received 17 February 2024; accepted 9 April 2024. Date of publication 18 April 2024; date of current version 4 September 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 92373207, and in part by the National Key Research and Development Program of China under Grant 2022YFB4400500 and Grant 2021YFA1003602. This brief was recommended by Associate Editor R. Venkatesan. (Corresponding author: Zhifeng Lin.)

Yilu Chen and Zhifeng Lin are with the Center for Discrete Mathematics and Theoretical Computer Science, Fuzhou University, Fuzhou 350108, China (e-mail: yiluchensolar@gmail.com; linzhifeng@fzu.edu.cn).

Xiaojian Liang is with the College of Computer and Data Science, Fuzhou University, Fuzhou 350108, China (e-mail: 211027075@fzu.edu.cn).

Zhijie Cai and Jianli Chen are with the School of Microelectronics, Fudan University, Shanghai 200433, China (e-mail: zjcai22@m.fudan.edu.cn; chenjianli@fudan.edu.cn).

Xiqiong Bai is with the School of Integrated Circuit Science and Engineering, Nanjing University of Posts and Telecommunications, Nanjing 210003, China (e-mail: 20230201@njupt.edu.cn).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TCSII.2024.3391004>.

Digital Object Identifier 10.1109/TCSII.2024.3391004

```

casez (s[2:0])
  3'b000: dout={din[1], din[2], din[3]};
  3'b?01: dout={1'b0, 1'b1, din[7]};
  3'b010: dout={din[1], ~din[5], din[6]};
  3'b011: dout={din[7], ~din[5], din[9]};
  default: dout={din[10], din[11], din[12]};
endcase

```

(a)

0	0	0	din[1]	din[2]	din[3]
-1	0	1	1'b0	1'b1	din[7]
0	1	0	din[1]	~din[5]	din[6]
0	1	1	din[7]	~din[5]	din[9]
			din[10]	din[11]	din[12]

(b)

(c)

Fig. 2. A CASE statement and its parsed matrices.

- Experimental results demonstrate that our proposed algorithm can achieve the best results among Yosys and the commercial tool Design Compiler.

The remainder of this brief is organized as follows. Section II presents the preliminaries of the problem. Section III first provides an overview of our algorithm and subsequently introduces the details. Section IV presents the experimental results, and we finally conclude in Section V.

II. PRELIMINARIES

In this brief, without loss of generality, we focus solely on the MUXes parsed from CASE statements. A MUXn has two sets of inputs, data inputs and address inputs. Moreover, a MUXn typically encompasses a default data input. Assume that the data inputs of the MUXn as $D = \{d_1, d_2, \dots, d_{n-1}, d_{def}\}$ and address inputs as $S = \{s_1, s_2, \dots, s_m\}$, where d_{def} is the default data input. Denote $A_i = s_1^{p_i^1} s_2^{p_i^2} \dots s_m^{p_i^m}$ as the address of the data input d_i , where $p_i^j \in \{-1, 0, 1\}$, $i \in \{1, 2, \dots, n-1\}$, $j \in \{1, 2, \dots, m\}$. When $p_i^j = -1$, it means that there is no necessity to employ this variable s_j in the address A_i , while $p_j = 0/1$ represents the phase assignment to variable s_j , i.e., $s_j^{-1} = 1$, $s_j^0 = \bar{s}_j$ and $s_j^1 = s_j$. Hence, the function of the portion of MUXn that does not include a default data input can be denoted as

$$f_{n-1} = A_1 d_1 + A_2 d_2 + \dots + A_{n-1} d_{n-1}. \quad (1)$$

Further, the function of the entire MUXn is formulated as

$$f(D, S) = f_{n-1} + (A_1 + A_2 + \dots + A_{n-1}) d_{def}. \quad (2)$$

A typical CASE statement is illustrated in Figure 2(a). A CASE statement consists of two parts: CASE items and their corresponding statements. The CASE items may include “don’t care” terms, denoted by “?”, which indicate that the value of that bit can be either 0 or 1. To facilitate the computation in our algorithm, we need to convert the CASE items into an address matrix, as shown in Figure 2(b), where the “don’t care” terms are represented as -1. Meanwhile, the statements are transformed into a data matrix representing the wires of data inputs. Figure 2(c) represents the data matrix of statements in Figure 2(a), where each row of the data matrix

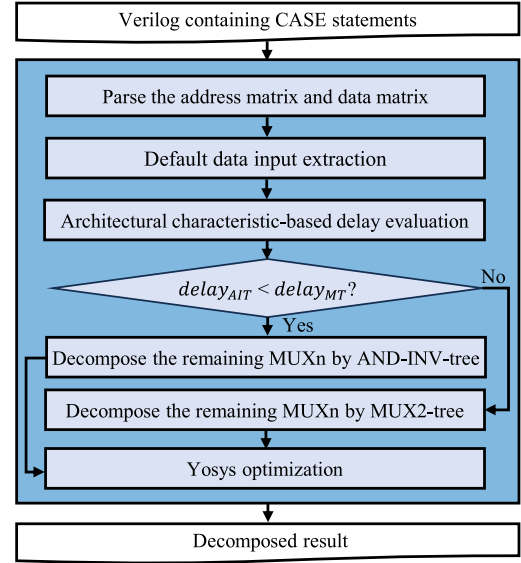


Fig. 3. Overall flow of our proposed algorithm.

corresponds to a row of the address matrix, and the additional last row corresponds to the default data inputs.

Moreover, in this brief, when the address input of MUX2 is 1, the selected data input is denoted as 1-data-input. Otherwise, it is denoted as 0-data-input.

III. ALGORITHM

The overall flow of our algorithm is illustrated in Figure 3. The details within the flow will be elaborated in the subsequent sections.

A. Default Data Input Extraction

Since the majority of MUXns contain default data inputs, we prioritize the separate treatment of default data inputs initially in order to simplify the complexity of the problem. First, we construct $n-1$ AND-INV-trees by dichotomy to represent each A_i , which means the logical AND combination of each variable in A_i . Second, we construct an OR-tree of the aforementioned AND-INV-trees by dichotomy. Thus, $A_1 + A_2 + \dots + A_{n-1}$ is constructed. Finally, we construct a MUX2, whose address input is the output of the aforementioned OR-tree, and 0-data-input is the default data input. Note that, according to De Morgan’s laws, the OR-tree can be converted to an AND-INV-tree. An example of the extraction approach is illustrated in Figure 4. Figure 4(a) illustrates the visualization of an original MUXn, while Figure 4(b) showcases the network after performing the extraction approach. In this network, the address input of the MUX2 is determined as $s[1]s[0] + \bar{s}[1] = (s[1]s[0])(\bar{s}[1])$, which effectively distinguishes the default data input $\{din[2]\}$ from the non-default data inputs $\{din[0], din[1]\}$.

B. AND-INV-Tree

Since the default data input is extracted by the above extraction approach, we only need to consider decomposing the remaining portion of MUXn that only contains non-default

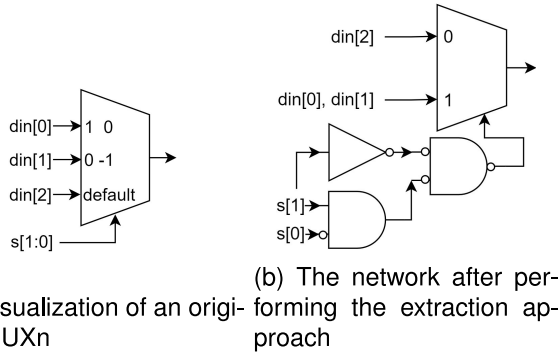


Fig. 4. The example of the extraction approach for the default data input.

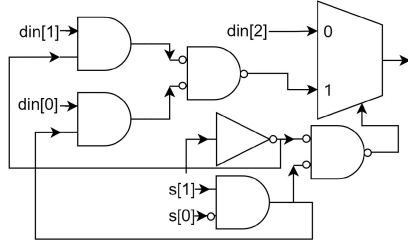


Fig. 5. The architecture of decomposing Figure 4(b) by AND-INV-tree.

data inputs. According to Equation (1) and De Morgan's laws, intuitively, it can be constructed only by INVs and ANDs to represent f_{n-1} . The procedures for constructing the remaining MUXn in the AND-INV tree are outlined as follows.

To reuse gates, first, we construct $n - 1$ AND-INV-tree by dichotomy to represent each address A_i . Second, we connect each A_i with its corresponding data input d_i using an AND to represent $A_i d_i$. Finally, we construct an OR-tree by dichotomy to represent f_{n-1} , whose leaves are the outputs of each $A_i d_i$. The output of f_{n-1} is connected to the 1-data-input of the MUX2 constructed by the extraction approach. The default decomposition method in Yosys also employs the same architecture. The result of decomposing Figure 4(b) into an AND-INV-tree is depicted in Figure 5. The final generated network is passed to Yosys for optimization.

C. MUX2-Tree

To construct a MUX2-tree, we will recursively select a decomposition bit from address inputs to partition the current data inputs into two sets. If the column of the selected decomposition bit does not include any “don't care” terms, the partitioning results in a set comprising 0-terms and another set comprising 1-terms. However, if the column of the decomposition bit includes “don't care” terms, the partitioning results in a set of “don't care” terms and a set of non-“don't care” terms. We denote the 0-term set and non-“don't care” term set as left set, and the 1-term set and “don't care” term set as right set.

Example 1: Consider the parsed address matrix and its corresponding data matrix in Figure 6(a). If we select $s[2]$ as the decomposition bit, as the column of $s[2]$ includes a “don't care” terms, the data inputs will be partitioned into a non-“don't care” term set, $\{din[0], din[1], din[2]\}$, and a

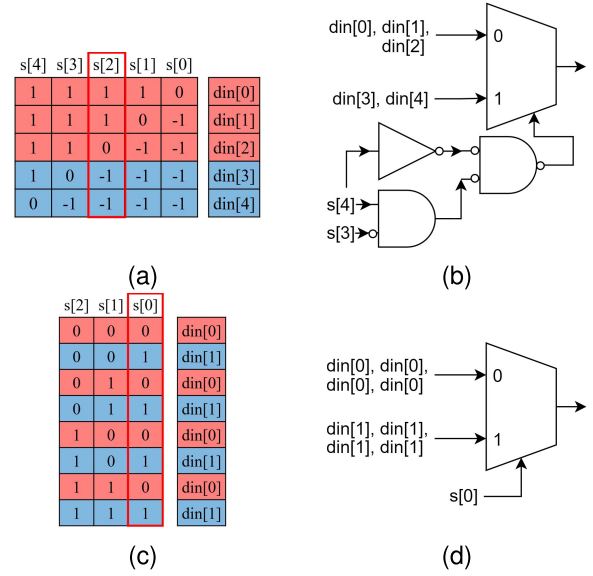


Fig. 6. The decomposition examples of constructing a MUX2-tree. (a) is a parsed address matrix and its corresponding data matrix. (b) is the architecture of (a) obtained by selecting $s[2]$ as the decomposition bit. (c) is another parsed address matrix and its corresponding data matrix. (d) is the architecture of (c) obtained by selecting $s[0]$ as the decomposition bit.

“don't care” term set, $\{din[3], din[4]\}$. The address input of the MUX2 is $s[4]s[3] + s[4] = (s[4]s[3])(s[4])$. The decomposed architecture is illustrated in Figure 6(b).

A decomposition bit s_i is better than another one s_j if one of the following five rules is satisfied:

- 1) The difference ($diff_{data}$) between the numbers of data inputs of the left set and right set decomposed by s_i is less than by s_j .
- 2) The number (num_{dc}) of “don't care” terms included in the column of s_i is less than in the column of s_j when Rule 1 does not apply.
- 3) The $areaReduce_i$ decomposed by s_i is greater than $areaReduce_j$ when Rule 1 and 2 do not apply, where the $areaReduce_i$ is an evaluation value to assess the reduction in area decomposed by s_i and is shown in Algorithm 1.
- 4) The difference ($diff_{pairs}$) between the numbers of pairs of same data inputs of the left set and right set decomposed by s_i is less than by s_j when Rule 1, 2, and 3 do not apply.
- 5) The difference ($diff_{inv}$) between the numbers of inverter terms of the left set and right set decomposed by s_i is greater than by s_j when previous rules do not apply.

To facilitate understanding, we provide one more example of constructing a MUX2-tree, as illustrated below.

Example 2: Consider the parsed address matrix and its corresponding data matrix in Figure 6(c). Assuming that the area of MUX2 is 2, the $diff_{data}$ obtained by sequentially selecting $\{s[2], s[1], s[0]\}$ is $\{0, 0, 0\}$, the num_{dc} obtained is $\{0, 0, 0\}$, and the $areaReduce$ obtained is $\{8, 8, 12\}$. Therefore, we will select $s[0]$ as the first decomposition bit for decomposition. The decomposed structure diagram is illustrated in Figure 6(d).

Constructing the MUX2-tree is a recursive algorithm that, when data inputs are partitioned into a left set and a right

TABLE I
EXPERIMENTAL RESULTS OF THE COMPARED ALGORITHMS AND OUR ALGORITHM, WHERE AF INDICATES THE AVERAGE
NUMBER OF FANOUTS AND SA REPRESENTS THE SELECTED DECOMPOSITION ARCHITECTURE OF OUR ALGORITHM

		Ours										Yosys [16] + ABC [17]										Design Compiler [19]										1st Place [20]			
		#ANDs	#MUX2s	#INVs	AF	Area	Delay	RT(s)	SA	#ANDs	#MUX2s	#INVs	AF	Area	Delay	RT(s)	#ANDs	#MUX2s	#INVs	AF	Area	Delay	RT(s)	Area	Delay	RT(s)									
casez	hb1	18	3	10	1.19	26.5	7.3	0.39	Aff	19	3	12	1.23	27.7	7.5	0.25	26	1	18	1.04	30.3	7.2	4.54	26.1	6.1	0.15									
	hb2	7	7	0	1.15	16.8	4.8	0.39	MT	19	3	12	1.23	28.0	6.8	0.21	23	2	19	1.04	33.4	5.6	4.62	20.2	4.3	0.14									
	hb3	7	7	0	1.15	21.0	4.2	0.37	MT	19	3	12	1.23	31.0	8.0	0.23	0	7	0	1.00	14.0	7.0	4.77	25.0	4.2	0.15									
constant	hb1	11	1	12	1.00	14.7	5.5	0.41	Aff	8	2	8	1.00	13.8	5.4	0.25	12	1	11	1.00	15.6	5.5	4.49	14.9	6.0	0.07									
	hb2	2	4	3	1.00	8.8	3.6	0.37	MT	8	2	8	1.00	14.0	4.9	0.21	14	1	9	1.00	19.0	4.7	4.01	10.6	3.6	0.06									
	hb3	1	5	0	1.00	11.0	3.0	0.35	MT	8	2	8	1.00	16.0	5.8	0.21	10	2	7	1.00	17.5	5.6	4.75	14.0	3.0	0.07									
large_casex	hb1	191	3	132	1.51	211.7	9.4	0.54	Aff	196	2	133	1.51	214.3	9.4	0.32	196	3	135	1.51	217.0	9.4	4.72	218.4	9.3	0.01									
	hb2	62	73	64	1.20	189.8	7.9	0.47	MT	196	2	133	1.51	252.0	8.5	0.31	222	2	131	1.53	277.2	8.3	4.86	234.6	8.4	0.01									
	hb3	60	81	80	1.21	247.0	8.4	0.48	MT	196	2	133	1.51	266.5	9.8	0.31	252	2	132	1.51	322.0	9.4	5.23	240.0	9.6	0.01									
large_mix	hb1	105714	9	51947	1.62	110931.2	20.3	266.45	Aff	109493	7	52656	1.63	114776.1	20.3	260.65	102410	57	51872	1.62	107739.7	20.4	3528.48	111827.4	20.3	4.50									
	hb2	24939	45284	20899	1.15	96696.2	18.0	76.13	MT	109493	7	52656	1.63	130565.2	18.3	201.09	101709	75	51766	1.62	122823.4	18.4	3377.90	113895.6	18.3	4.43									
	hb3	17966	53205	10129	1.14	129440.5	20.4	82.62	MT	109493	7	52656	1.63	135835.0	20.6	313.78	101899	50	51824	1.62	127911.0	20.8	2745.99	114588.0	20.6	4.46									
mix	hb1	2	3	2	1.00	9.7	5.2	0.38	Aff	2	3	2	1.00	9.7	5.2	0.23	2	3	2	1.00	9.7	5.2	4.59	13.5	6.1	0.04									
	hb2	0	4	1	1.00	6.0	3.7	0.35	MT	2	3	2	1.00	7.0	3.5	0.21	2	3	2	1.00	7.0	3.5	4.10	10.6	3.7	0.04									
	hb3	0	4	1	1.00	8.5	3.2	0.41	MT	2	3	2	1.00	9.0	3.4	0.21	2	3	2	1.00	9	3.4	4.36	13.5	3.2	0.04									
multi_bits	hb1	62	0	46	1.37	66.6	6.3	0.40	Aff	62	0	44	1.39	66.4	6.3	0.25	52	4	49	1.31	66.9	6.6	4.61	76.9	6.3	0.42									
	hb2	3	23	11	1.00	39.6	3.7	0.39	MT	62	0	44	1.39	79.6	5.7	0.24	53	3	47	1.32	76.0	5.9	4.54	56.2	4.5	0.43									
	hb3	2	24	9	1.00	54.5	3.2	0.42	MT	62	0	44	1.39	84.0	6.6	0.24	53	3	47	1.32	82.5	7.0	4.75	72.5	4.6	0.44									
normal	hb1	0	7	0	1.00	17.5	6.0	0.37	Aff	0	7	0	1.00	17.5	6.0	0.31	0	7	0	1.00	17.5	6.0	4.61	17.6	6.0	0.08									
	hb2	0	7	0	1.00	9.8	3.6	0.23	MT	0	7	0	1.00	9.8	3.6	0.23	0	7	0	1.00	9.8	3.6	4.64	10.2	3.6	0.08									
	hb3	0	7	0	1.00	14.0	3.0	0.24	MT	0	7	0	1.00	14.0	3.0	0.21	0	7	0	1.00	14.0	3.0	4.72	14.5	3.0	0.08									
not	hb1	0	7	1	1.00	17.6	6.1	0.38	Aff	0	7	4	1.00	17.9	6.1	0.25	0	7	3	1.00	17.8	6.1	4.52	17.8	6.2	0.08									
	hb2	0	7	1	1.00	10.2	3.7	0.37	MT	0	7	4	1.00	11.4	3.7	0.21	0	7	2	1.00	10.6	3.7	4.55	11.0	3.8	0.08									
	hb3	0	7	1	1.00	14.5	3.2	0.35	MT	0	7	4	1.00	16.0	3.2	0.22	0	7	4	1.00	16.0	3.2	4.70	15.5	3.4	0.08									
same_chosen	hb1	0	3	0	1.00	7.5	4.0	0.23	Aff	0	3	0	1.00	7.5	4.0	0.23	0	3	0	1.00	7.5	4.0	4.62	7.6	4.0	0.04									
	hb2	0	3	0	1.00	4.2	2.4	0.23	MT	0	3	0	1.00	4.2	2.4	0.21	0	3	0	1.00	4.2	2.4	4.49	4.6	2.4	0.04									
	hb3	0	3	0	1.00	6.0	2.0	0.24	MT	0	3	0	1.00	6.0	2.0	0.22	0	3	0	1.00	6.0	2.0	4.71	6.5	2.0	0.04									
Normalized		1.00	1.00	1.00	1.00	1.00	1.00	1.00	-	2.21	0.001	1.90	1.09	1.19	1.17	0.87	2.06	0.003	1.87	1.06	1.19	1.15	15.23	1.16	1.04	0.28									

Algorithm 1 Calculate *areaReduce*

Require: left set, right set, MUX2 area a_{mux2} , AND area a_{and} , INV area a_{inv} , whether to transform MUX2 to OR *transform_to_or*

Ensure: *areaReduce*

```

1: Initialization: areaReduce  $\leftarrow$  0
2: for each set in {left set, right set} do
3:   map M  $\leftarrow$  count the number of each data input in set
4:   for each item in M do
5:     if item.first is 1'd0 then
6:       areaReduce  $+= a_{mux2} - a_{and}$ 
7:     end if
8:     if item.first is 1'd1 & transform_to_or then
9:       areaReduce  $+= a_{mux2} - a_{and} - a_{inv} \times 3$ 
10:    end if
11:    areaReduce  $+= a_{mux2} \times (item.second - 1)$ 
12:  end for
13:  if both 1'd0 and 1'd1 are in M then
14:    if transform_to_or then
15:      areaReduce  $+= a_{and} \times 2 + a_{inv} \times 3 - a_{mux2}$ 
16:    else
17:      areaReduce  $+= a_{and}$ 
18:    end if
19:  end if
20: end for

```

set, recursively applies the same method to further decompose each set until there are only two data inputs or one data input remaining. In the case of two remaining data inputs, an unused address input that can distinguish between them is selected to perform the decomposition. Subsequently, we connect the output of the generated MUX2-tree with the 1-data-input of the MUX2 of the extraction approach. Finally, we pass the generated architecture, excluding MUX2s, to Yosys for optimization.

D. Architectural Characteristic-Based Delay Evaluation

With two decomposition architectures available, it is necessary to determine which architecture to use. Firstly, we

evaluate the optimal delay of an AND-INV-tree using a dichotomy as follows.

$$delay_{AIT} = (\log_2^m + \log_2^{n-1} + 1) \times dly_{and} + 2 \times dly_{inv}, \quad (3)$$

where m is the size of address input set S and $n-1$ means the number of data inputs except the default data input. Moreover, dly_{and2} and dly_{inv} represents the delay of AND and INV, respectively.

As shown in Figure 2(a), a CASE statement may contain multiple outputs, which implies that a CASE statement can include multiple MUXNs. We define the set of outputs as $O = \{o_1, o_2, \dots, o_h\}$, where h represents the number of outputs. Correspondingly, these outputs' data input sets are denoted as $D_1, D_2, \dots, D_h$, respectively. Due to the possibility of duplicate data inputs within each data input set, we remove the duplicated data inputs and the default data input and denote the resulting non-default data input set as, for example, $D_i^{unique} = \{d_1^i, d_2^i, \dots, d_u^i\}$, where $u \leq n-1$, with corresponding counts denoted as $NUM_i = \{num_1^i, num_2^i, \dots, num_u^i\}$. Hence, secondly, we evaluate the optimal delay of a MUX2-tree using a dichotomy as follows.

$$delay_{MT} = \log_2^{\max_{i \in [1, \dots, h]} n_i^{unique}} \times dly_{mux2}, \quad (4)$$

$$n_i^{unique} = \sum_{j \in [1, \dots, u]} (1 + num_j^i > 1 ? num_j^i \% 2 : 0), \quad (5)$$

where dly_{mux2} represents the delay of MUX2. The reason behind designing the evaluated $delay_{MT}$ in this manner is to account for the potential merging of identical data inputs, which can reduce the area and delay of the MUX2-tree, as shown in Figure 6(d).

As depicted in Figure 3, if $delay_{AIT} < delay_{MT}$, we select the AND-INV-tree to decompose the remaining MUXn after the extraction approach. Otherwise, the MUX2-tree is selected.

IV. EXPERIMENTAL RESULTS

Our proposed algorithm is implemented in C++. The experiments were performed on Ubuntu 20.04 running on 2.42GHz Intel Core with 16GB memory, using the benchmarks provided by the 2022 integrated circuit EDA design elite

TABLE II
THE AREA AND DELAY OF DIFFERENT BASIC GATES

Cell	lib1		lib2		lib3	
	Area	Delay	Area	Delay	Area	Delay
MUX2	2.5	2	1.4	1.2	2	1
AND	1	1	1	0.9	1	1
INV	0.1	0.1	0.4	0.1	0.5	0.2

challenge [21], which was held by the Chinese Institute of Electronics. In these benchmarks, the size of the address input sets ranges from 3 to 20, representing numbers of addresses from 8 to $2^{20} = 1048576$.

The experiments utilized only three types of basic logic gates, namely INV, AND, and MUX2. Their respective area and delay in different libraries are presented in Table II.

In Table I, we compare the quality and runtime(RT) of our algorithm with Yosys [16]+ABC [17], the commercial tool Design Compiler [19], and the 1st place [20] of the 2022 integrated circuit EDA design elite challenge. The Yosys script of Yosys+ABC is as follows: `read_verilog; hierarchy; proc; techmap; opt; abc; opt; write_verilog`. The ABC script of it is as follows: `resyn2; &get -n; &dch -f; &nf; &put`. Due to the requirement imposed by Design Compiler for the inclusion of an OR gate, the Design Compiler library comprises four distinct cells: AND, INV, MUX2, and OR. It is noteworthy that the OR gate is considered as comprising one AND gate and three INV gates. The Yosys and ABC scripts utilized in our algorithm for further optimization are the same as Yosys+ABC.

As shown in Table I, our algorithm demonstrates superior performance in terms of area and delay for most examples. Specifically, compared to Yosys+ABC, our algorithm achieves a reduction of 19% in area and 17% in delay. In comparison to Design Compiler, our algorithm achieves a reduction of 19% in area and 15% in delay. Furthermore, when compared to the 1st place, our algorithm achieves a reduction of 16% in area and 4% in delay. From the table, it can be seen that Yosys+ABC and Design Compiler tend to employ a greater number of AND gates. Therefore, when decomposing into MUX2-trees is better, their results are comparatively inferior. Conversely, our algorithm has the flexibility to select between decomposing into AND-INV-trees or MUX2-trees based on the characteristics of the design. Furthermore, the generation of the AND-INV-tree involves significantly sharing AND gates. As a result, the average fanout number for Design Compiler and Yosys tends to exceed that of our algorithm. In most examples, our algorithm exhibits longer runtime compared to Yosys+ABC and the 1st place. This is due to our algorithm, which involves writing out the network and sending it to the Yosys tool for optimization, resulting in additional time overhead. However, by embedding our algorithm directly into the Yosys tool, we believe that this time overhead can be eliminated.

V. CONCLUSION

This brief proposed a network-architecture-aware multiplexer decomposition that employs two distinct network

architectures: AND-INV-tree and MUX2-tree. We designed an architectural characteristic-based delay evaluation scheme to determine the superior decomposition architecture for decomposing. Experimental results demonstrated the superiority of our algorithm over Yosys, Design Compiler, and the 1st place in the competition. In future work, we will consider more complex gates, such as MUX4 and MUX8, for use in multiplexer decomposition and consider the fanout number when calculating the delay of gates.

REFERENCES

- [1] K. Tu, X. Tang, C. Yu, L. Josipović, and Z. Chu, "Logic synthesis," in *FPGA EDA: Design Principles and Implementation*. Singapore: Springer, 2024, pp. 135–164.
- [2] S. Thakur, D. F. Wong, and S. Krishnamoorthy, "Delay minimal decomposition of multiplexers in technology mapping," in *Proc. Conf. Design Autom.* 1996, pp. 254–257.
- [3] J. Cong and Y. Ding, "Combinational logic synthesis for LUT based field programmable gate arrays," *ACM Trans. Design Autom. Electron. Syst.*, vol. 1, no. 2, pp. 145–204, 1996.
- [4] Y. Jiang, A. Al-Sheraidah, Y. Wang, E. Sha, and J.-G. Chung, "A novel multiplexer-based low-power full adder," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 51, no. 7, pp. 345–348, Jul. 2004.
- [5] U. S. Kumar, K. M. Salih, and K. Sajith, "Design and implementation of carry select adder without using multiplexers," in *Proc. Int. Conf. Emerg. Technol. Trends Electron., Commun. Netw.*, 2012, pp. 1–5.
- [6] N. K. Singh and P. K. Sharma, "2T 2: 1 MUX based 1 bit full adder design," in *Proc. Int. Conf. Commun. Signal Process.* 2014, pp. 1491–1493.
- [7] S. Murugeswari and S. K. Mohideen, "Design of area efficient and low power multipliers using multiplexer based full adder," in *Proc. Int. Conf. Current Trends Eng. Technol.*, 2014, pp. 388–392.
- [8] K. Z. Pekmestzi, "Multiplexer-based array multipliers," *IEEE Trans. Comput.*, vol. 48, no. 1, pp. 15–23, Jan. 1999.
- [9] K. Wang, S. Chen, W. Cao, L. Wang, and J. Tong, "A coarse-grained reconfigurable computing unit," in *Proc. IEEE Int. Conf. (ASIC)*, 2011, pp. 87–90.
- [10] C. Arnold, J. Parlebas, R. Meiser, and T. Zwick, "Fully reconfigurable manifold multiplexer," *IEEE Trans. Microw. Theory Techn.*, vol. 65, no. 10, pp. 3885–3891, Oct. 2017.
- [11] N. Das and P. A. Priya, "FPGA implementation of reconfigurable finite state machine with input multiplexing architecture using Hungarian method," *Int. J. Reconfig. Comput.*, vol. 2018, pp. 1–15, Jan. 2018.
- [12] E. J. McCluskey, *Logic Design Principles with Emphasis on Testable Semicustom Circuits*. Upper Saddle River, NJ, USA: Prentice-Hall Inc., 1986.
- [13] S. Mitra, L. J. Avra, and E. J. McCluskey, "Efficient multiplexer synthesis techniques," *IEEE Design Test Comput.*, vol. 17, no. 4, pp. 90–97, Dec. 2000.
- [14] C. Vudadha, A. Surya, S. Agrawal, and M. B. Srinivas, "Synthesis of ternary logic circuits using 2:1 multiplexers," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 12, pp. 4313–4325, Dec. 2018.
- [15] H. Fang, Z. Ma, F. Yu, B. Zhao, and B. Zhang, "Optimised serial commutator FFT architecture in terms of multiplexers," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 1, pp. 445–449, Jan. 2024.
- [16] C. Wolf, J. Glaser, and J. Kepler, "Yosys-a free verilog synthesis suite," in *Proc. Austrian Workshop Microelectron.*, 2013, p. 97.
- [17] R. Brayton and A. Mishchenko, "ABC: An academic industrial-strength verification tool," in *Proc. 22nd Int. Conf. Comput. Aided Verificat.* 2010, pp. 24–40.
- [18] M. M. A. Basiri and S. N. Mahammad, "High speed multiplexer design using tree based decomposition algorithm," *Microelectron. J.*, vol. 51, pp. 99–111, May 2016.
- [19] "Design compiler." Synopsys. 2023. [Online]. Available: <https://www.synopsys.com>
- [20] Z. Wang, L. Chen, S. Wang, J. Zhou, C. Tian, and H. Feng, "A synthesis method for verilog case statement using MUX-and-inverter graph," in *Proc. Int. Symp. Electron. Design Autom.*, 2023, pp. 174–178.
- [21] (Chin. Inst. Electron., Beijing, China). *The Integrated Circuit EDA Design Elite Challenge*. (2022). <https://eda.icisc.cn/en/>